

ACM CCS '26 Artifact Appendix: Boosting Query Recovery against Encrypted Databases: High-Accuracy and Efficient Inference Attacks Using Minimal Leakage

Jiaojiao Wu*
School of Cryptology
Xidian University
School of Artificial Intelligence and Computer Science
Shaanxi Normal University
Xi'an, China
jjwu@snnu.edu.cn

Lei Zhang*
School of Cryptology
Xidian University
Xi'an, China
vhzn@foxmail.com

Jianfeng Wang†
School of Cryptology
Xidian University
Xi'an, China
jfwang@xidian.edu.cn

Shi-Feng Sun
School of Computer Science
Shanghai Jiao Tong University
Shanghai, China
shifeng.sun@sjtu.edu.cn

Xiaofeng Chen
State Key Laboratory of Integrated Service Networks (ISN)
Xidian University
Xi'an, China
xfchen@xidian.edu.cn

A Artifact Appendix

This artifact appendix accompanies the paper "Boosting Query Recovery against Encrypted Databases: High-Accuracy and Efficient Inference Attacks Using Minimal Leakage." It describes the contents of the artifact and provides step-by-step instructions for reproducing the main experiments and figures of the paper.

A.1 Abstract

The artifact is a Python 3.9 reference implementation of Midas, a high-accuracy and efficient query recovery attack against encrypted databases that relies on minimal leakage. It includes the source code of Midas and its lightweight variant IHOP^{M}, which augments the IHOP attack with iterative refinement, together with five baseline attacks (Jigsaw, IHOP, IKK, SAP, and Score), preprocessed statistics of the Enron and Lucene datasets, experiment drivers, and plotting scripts. Each experiment script runs from the repository root, writes intermediate results to `pic_pkl/`, and its matching plotting script (run from `pic/`) exports the corresponding publication-ready figure, covering parameter studies, ablation studies, comparisons with prior attacks, and evaluations against the CLRZ and OSSE defenses. The artifact is released under the MIT license; a maintained version is available on GitHub and an archived version on Zenodo.

A.2 Description & Requirements

The artifact is distributed as a self-contained repository. The `README.md` at the repository root is the main entry point; it documents the repository layout, environment setup, and the exact commands for every experiment. This section summarizes the accessibility, dependencies, and benchmarks of the artifact.

*Co-first authors.
†Corresponding author.

A.2.1 Security, privacy, and ethical concerns. None. The artifact implements inference attacks that operate on simulated leakage derived from publicly available datasets. Executing the artifact requires no network access, no elevated privileges, and no destructive or intrusive actions, so it poses no security, privacy, or ethical risk to the evaluator.

A.2.2 How to access. The maintained version of the artifact is available on GitHub at https://github.com/CDSecLab/Midas. A permanently archived copy is available on Zenodo at https://doi.org/10.5281/zenodo.22692003.

A.2.3 Hardware dependencies. All experiments were executed on a 64-bit machine equipped with an Intel i5-11500 CPU (12 cores, 2.7 GHz) and 16 GB of RAM; this serves as the reference configuration for the time estimates reported in the paper. The runtime of the experiments is primarily determined by the CPU frequency, and at least 16 GB of RAM is required. Matching the exact CPU model is not necessary: any 64-bit x86 machine with comparable or higher CPU frequency runs the experiments correctly, typically faster. No GPU or other specialized hardware is required.

A.2.4 Software dependencies. The artifact targets Python 3.9 and depends on the following packages: `numpy`, `pandas`, `scipy`, `matplotlib`, `seaborn`, and `tqdm`. All dependencies are installed with `pip3 install -r requirements.txt`. The operating system is not restricted; the artifact is platform independent.

A.2.5 Benchmarks. The experiments use two publicly available datasets, Enron and Lucene, together with three query-generation scenarios S1–S3 defined in the paper. Preprocessed keyword and query statistics are already included in `Datasets/` (`Enron_3000.pkl`, `Lucene_3000.pkl`, and `Lucene_6000.pkl`), so no additional downloading or preprocessing is required for reproduction.

A.3 Set Up

This section summarizes the steps to install the artifact and verify that the environment is working. Detailed instructions are available in the `README.md` of the artifact.

A.3.1 Installation. Clone the repository from GitHub at https://github.com/CDSecLab/Midas or extract the archived Zenodo version, then install the dependencies from the repository root:

```
pip3 install -r requirements.txt
```

The artifact targets Python 3.9; see Section A.2 for the list of dependencies and the reference hardware configuration.

A.3.2 Basic test. Run the Figure 2 experiment on the Enron dataset as an end-to-end functionality test. First, from the repository root, run the experiment script; it executes the PR parameter study and writes `pic_pkl/PREnron.pkl`:

```
python PRTest.py -d Enron
```

Then, from the `pic/` directory, generate the figure:

```
cd pic
python PRaPlot.py -d Enron
python PRbPlot.py -d Enron
```

Expected successful output: two PDFs, `PRaEnron.pdf` and `PRbEnron.pdf`, are produced in `pic/pictures/`.

A.4 Evaluation Workflow

All experiments follow the same workflow: an experiment script is run from the repository root and writes its intermediate results to `pic_pkl/`; the matching plotting script is then run from `pic/` and exports the final figure into `pic/pictures/`. The complete list of commands for every experiment is given in `README.md`.

A.4.1 Major claims.

(C1): Midas recovers queries with high accuracy under minimal leakage, being among the attacks with the highest recovery accuracy. Over varying auxiliary document sizes (E3, Figure 9), keyword-universe sizes (E4, Figures 10–11), and numbers of observed queries (E5, Figures 12–13), this high accuracy is maintained. In particular, under a similar attack time its accuracy is higher than that of the other attacks (Figure 11), and at large keyword-universe sizes the advantage is more pronounced, being slightly above the other strong attacks (Figure 10).

(C2): Midas strikes a favorable balance between efficiency and accuracy: most of the baselines that run faster than Midas achieve lower accuracy, whereas those whose accuracy is comparable to Midas run slower, with Midas retaining a small accuracy edge as well. Its running time grows roughly linearly in the number of observed queries. Furthermore, Midas components can also improve other attacks: IHOP^{M}, i.e., IHOP augmented with Midas's iterative refinement, converges to comparable accuracy within fewer iterations than IHOP. This is supported by experiments E1, E4, and E5 (Figure 5 of Section 4.2.2 and Figures 10–13 of Sections 4.4.2–4.4.3).

(C3): Each component of Midas plays an essential role: the co-absence leakage of prior queries and the iterative refinement using the refined ranking are necessary for its accuracy, while the incremental recomputation of object candidates reduces the running time of Midas itself. This is supported by experiment E2 (Figures 6–8 of Section 4.3).

(C4): Midas remains highly effective against the CLRZ and OSSE query-based defenses. This is supported by experiment E6 (Figures 14–15 of Appendix A).

A.4.2 Experiments. We provide two ways to reproduce the experiments: (1) batch scripts for reproducing a group of figures in one shot, and (2) individual commands for reproducing a specific figure or executing the underlying steps separately.

Experiment scripts run from the repository root; plotting scripts run from the `pic/` directory. To reproduce all figures of the paper in one shot, run the four batch scripts from the repository root (one per figure group):

```
bash scripts/run_parameter.sh    # E1, Figures 2-5
bash scripts/run_ablation.sh     # E2, Figures 6-8
bash scripts/run_comparison.sh   # E3-E5, Figures 9-13
bash scripts/run_defense.sh      # E6, Figures 14-15
```

The following individual commands can be used to reproduce a specific figure. The reported time estimates are indicative and based on the reference machine described in Section A.2.

The short flags `-d` and `-s` abbreviate `–dataset` and `–scenarios`. In the command blocks below, a brace group such as `{Enron,Lucene}` or `{S1,S2,S3}` enumerates the alternative arguments: run the command once per value, choosing exactly one. For instance, `python auxTest.py -d {Enron,Lucene} -s {S1,S2,S3}` is executed once for each of the six dataset/scenario pairs (`Enron/S1`, `Enron/S2`, `Enron/S3`, `Lucene/S1`, `Lucene/S2`, `Lucene/S3`). The loop below runs all pairs in one shot; substituting script names and value lists reproduces any other block:

```
for d in Enron Lucene; do for s in S1 S2 S3; do
  python auxTest.py -d $d -s $s
  ( cd pic && python auxPlot.py -d $d -s $s )
done; done
```

(E1): **[Parameter selection] [15 person-minutes; ~3.5 hours]**: Reproduce the parameter studies (Figures 2–5, Sections 4.2.1–4.2.2). **Preparation.** The PR run is identical to the Basic test of Section A.3. **Execution**:

```
# Fig. 2: PR study
python PRTest.py -d Enron
python PRaPlot.py -d Enron
python PRbPlot.py -d Enron
# Fig. 3: RR study
python RRTest.py -d Enron
python RRPlot.py -d Enron
# Fig. 4: CR study
python CRTest.py -d Enron
python CRPlot.py -d Enron
# Fig. 5: IHOP^M study
python ihopMTest.py -d Enron
python ihopMPlot.py -d Enron
```

Results. The accuracy curves corroborate the parameter choices in the paper, and IHOP^{M} converges to comparable

accuracy within fewer iterations than IHOP, supporting C2.

(E2): **[Ablations] [10 person-minutes + 1 compute-hour]**: Check the contribution of each component (Figures 6–8, Section 4.3). **Preparation.** Environment as in Section A.3 (dataset defaults to Enron). **Execution** (Figures 6 and 7 run for both S1 and S2; the Figure 8 scripts take no scenario argument):

```
# Fig. 6: co-absence leakage
python ablationLeakageTest.py -s {S1,S2}
python ablationLeakagePlot.py -s {S1,S2}
# Fig. 7: iterative refinement
python ablationRRTest.py -s {S1,S2}
python ablationRRPlot.py -s {S1,S2}
# Fig. 8: incremental computation
python ablationOptimizeMTest.py
python ablationOptimizeNTest.py
python ablationOptimizeMPlot.py
python ablationOptimizeNPlot.py
```

Results. Dropping the co-absence leakage degrades the accuracy of Midas; omitting the iterative refinement degrades the accuracy likewise, and omitting the incremental recomputation increases the running time, supporting C3.

(E3): **[Auxiliary document size comparison] [20 person-minutes + 14 compute-hours]**: Vary the size of the simulated auxiliary document and compare the accuracy of all seven attacks (Figure 9, Section 4.4.1). **Preparation.** Environment as in Section A.3. **Execution** (each of the six dataset/scenario pairs costs ≈2.5 hours):

```
# Fig. 9: auxiliary document size
python auxTest.py -d {Enron,Lucene} -s {S1,S2,S3}
python auxPlot.py -d {Enron,Lucene} -s {S1,S2,S3}
```

Results. Six pkls in `pic_pkl/` and six figures in `pic/pictures/` composing Figure 9; the high-accuracy attacks (Midas, IHOP, IHOP^{M}, Score) reach comparable accuracy, supporting C1.

(E4): **[Keyword-universe comparison] [20 person-minutes + 10 compute-hours]**: Vary the keyword universe size (Figures 10–11, Section 4.4.2). **Preparation.** Environment as in Section A.3. **Execution** (Figure 10 on Enron for S1–S3; Figure 11 on Enron and Lucene for S1 and S2, run again with `-d Lucene` and with `-s S2`):

```
# Fig. 10: keyword universe size
python nTest.py -d Enron -s {S1,S2,S3}
python nPlot.py -d Enron -s {S1,S2,S3}
python nTimePlot.py -d Enron -s {S1,S2,S3}
# Fig. 11: similar runtime
python nTestLimitedTime.py -d {Enron,Lucene} -s {S1,S2}
python limitedTimeNPlot.py -d {Enron,Lucene} -s {S1,S2}
```

Results. Accuracy against the keyword universe size for all attacks, and their running times (Figure 10), where the attacks of accuracy comparable to Midas run slower than it. Under a fixed time budget (Figure 11), Midas and IHOP^{M} achieve higher accuracy than the other attacks in most cases, supporting C1 and C2.

(E5): **[Query-number comparison] [15 person-minutes + 11 compute-hours]**: Vary the number of observed queries (Figures 12–13, Section 4.4.3). **Preparation.** Environment as in Section A.3. **Execution** (Figure 12 on Enron for S1–S3; Figure 13 on Enron and Lucene for S1 and S2, as in its n counterpart E4):

```
# Fig. 12: number of observed queries
python mTest.py -d Enron -s {S1,S2,S3}
python mPlot.py -d Enron -s {S1,S2,S3}
python mTimePlot.py -d Enron -s {S1,S2,S3}
# Fig. 13: similar runtime
python mTestLimitedTime.py -d {Enron,Lucene} -s {S1,S2}
python limitedTimeMPlot.py -d {Enron,Lucene} -s {S1,S2}
```

Results. Accuracy against the number of observed queries for all attacks (Figure 12), where Midas runs faster than the attacks of comparable accuracy; its runtime grows roughly linearly. Under a fixed time budget (Figure 13) the attacks achieve comparable accuracies, supporting C1 and C2.

(E6): **[Defenses] [20 person-minutes + 18 compute-hours]**: Check the accuracy of Midas against the CLRZ and OSSE defenses (Figures 14–15, Appendix A). **Preparation.** Environment as in Section A.3. **Execution** (each of the six dataset/scenario pairs costs ≈1.5 hours per pair per defense):

```
# Fig. 14: CLRZ defense
python clrzTest.py -d {Enron,Lucene} -s {S1,S2,S3}
python clrzPlot.py -d {Enron,Lucene} -s {S1,S2,S3}
# Fig. 15: OSSE defense
python osseTest.py -d {Enron,Lucene} -s {S1,S2,S3}
python ossePlot.py -d {Enron,Lucene} -s {S1,S2,S3}
```

Results. Midas retains high attack accuracy under both the CLRZ and OSSE defenses, i.e., the defenses do not neutralize it, supporting C4.

In all of the above blocks, a successful reproduction yields figures that match the corresponding figures in the paper. Some experiments use random draws, so outputs may fluctuate slightly between runs, but the trends should match the paper.

A.5 Notes on Reusability

Readers can apply the pipeline to their own data by placing a pickle in `Datasets/` that follows the same four-element format as `Enron_3000.pkl`; all experiment and plotting scripts accept it through the `-d` flag. The scripts that build the provided datasets from the raw data are included in `preprocess/`, and SHA-256 checksums are listed in the `README.md`.

A.6 Version

Based on the LaTeX template for Artifact Evaluation V20220926.